

PL/SQL Exercises

Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.

- Question: Write a stored procedure `ProcessMonthlyInterest` that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

SQL Command :

```
mysql> DELIMITER $$
```

```
mysql>
```

```
mysql> CREATE PROCEDURE ProcessMonthlyInterest()
```

```
-> BEGIN
```

```
->
```

```
-> UPDATE Accounts
```

```
-> SET Balance = Balance + (Balance * 0.01)
```

```
-> WHERE AccountType = 'Savings';
```

```
->
```

```
-> END$$
```

Query OK, 0 rows affected (0.02 sec)

```
mysql>
```

```
mysql> DELIMITER ;
```

```
mysql> CALL ProcessMonthlyInterest();
```

Query OK, 1 row affected (0.03 sec)

```
mysql> SELECT AccountID, CustomerID, AccountType, Balance
```

```
-> FROM Accounts;
```

```
+-----+-----+-----+-----+
| AccountID | CustomerID | AccountType | Balance |
+-----+-----+-----+-----+
|      1 |      1 | Savings    | 1010.00 |
|      2 |      2 | Checking   | 1500.00 |
+-----+-----+-----+-----+
```

2 rows in set (0.00 sec)

NOTE :

The `ProcessMonthlyInterest` stored procedure applies a monthly interest rate of 1% to all savings accounts and updates their balances automatically.

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

Question: Write a stored procedure **`UpdateEmployeeBonus`** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

SQL Command:

```
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE UpdateEmployeeBonus(
  ->   IN dept_name VARCHAR(50),
  ->   IN bonus_percent DECIMAL(5,2)
  -> )
  -> BEGIN
  ->
  ->   UPDATE Employees
  ->   SET Salary = Salary + (Salary * bonus_percent / 100)
  ->   WHERE Department = dept_name;
  ->
  -> END$$
Query OK, 0 rows affected (0.01 sec)
```

```
mysql>
mysql> DELIMITER ;
mysql> CALL UpdateEmployeeBonus('IT', 10);
Query OK, 1 row affected (0.02 sec)
```

```
mysql> CALL UpdateEmployeeBonus('HR', 5);
Query OK, 1 row affected (0.00 sec)
```

```
mysql> SELECT EmployeeID, Name, Department, Salary
  -> FROM Employees;
+-----+-----+-----+-----+
| EmployeeID | Name      | Department | Salary |
+-----+-----+-----+-----+
|      1 | Alice Johnson | HR      | 73500.00 |
|      2 | Bob Brown    | IT      | 66000.00 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

NOTE:

-The UpdateEmployeeBonus stored procedure updates employee salaries by applying a user-defined bonus percentage to all employees within a specified department.

Scenario 3: Customers should be able to transfer funds between their accounts.

Question: Write a stored procedure TransferFunds that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

SQL Command:

```
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE TransferFunds(
  ->   IN from_account INT,
  ->   IN to_account INT,
```

```

-> IN transfer_amount DECIMAL(10,2)
-> )
-> BEGIN
-> DECLARE source_balance DECIMAL(10,2);
->
-> SELECT Balance
-> INTO source_balance
-> FROM Accounts
-> WHERE AccountID = from_account;
->
-> IF source_balance >= transfer_amount THEN
->
->     UPDATE Accounts
->     SET Balance = Balance - transfer_amount
->     WHERE AccountID = from_account;
->
->     UPDATE Accounts
->     SET Balance = Balance + transfer_amount
->     WHERE AccountID = to_account;
->
-> ELSE
->
->     SELECT 'Insufficient Balance. Transfer Cancelled.' AS Message;
->
-> END IF;
->
-> END$$

```

Query OK, 0 rows affected (0.01 sec)

```

mysql>
mysql> DELIMITER ;
mysql> CALL TransferFunds(1, 2, 500);
Query OK, 1 row affected (0.01 sec)

```

```

mysql> SELECT AccountID, CustomerID, Balance
-> FROM Accounts;

```

```

+-----+-----+-----+
| AccountID | CustomerID | Balance |
+-----+-----+-----+
|      1 |      1 | 510.00 |
|      2 |      2 | 2000.00 |
+-----+-----+-----+

```

2 rows in set (0.00 sec)

NOTE:

-The TransferFunds stored procedure securely transfers funds between accounts after validating that the source account has sufficient balance to complete the transaction.